CS 207: Design and Analysis of Algorithms

Practice Questions

1. Given two matrices A and B of orders $n \times n$, design a divide-and-conquer algorithm to find the product of the matrices in $O(n^3)$ time. Can you improve the running time to $O(n^{2.81})$?

2. Given two strings X of length m and Y of length n, a subsequence is a sequence obtained by deleting zero or more characters without changing the order. The **Longest Common Subsequence (LCS)** of X and Y is the longest sequence that is a subsequence of both strings.

   Using Dynamic Programming, define a suitable DP state, derive the recurrence relation, specify the base cases and analyze the time complexity.

3. Let $T = (V, E)$ be a tree with n vertices. A *Vertex Cover* of T is a subset $C \subseteq V$ such that every edge in E has at least one endpoint in C. A *Dominating Set* of T is a subset $D \subseteq V$ such that every vertex in V is either in D or adjacent to a vertex in D.

   Design dynamic programming algorithms on trees to compute:

   - The size of a minimum vertex cover of T.
   - The size of a minimum dominating set of T.

   For each problem: root the tree at an arbitrary node, define appropriate DP states for each node, derive the recurrence relations, specify base cases and nalyze the time complexity.

4. You are given a cache that can hold at most k pages and a sequence of page requests $r_1, r_2, \ldots, r_n$.

   When a requested page is already in the cache, it is a *hit*. If it is not in the cache, it is a *miss*, and the page must be loaded into the cache. If the cache is full, one page must be evicted before loading the new page.

   Assume the entire request sequence is known in advance (offline setting). Answer the following:

   (a) Design a greedy algorithm to minimize the total number of cache misses.

   (b) Clearly state the eviction rule used by your algorithm.

   (c) Prove that your greedy strategy is optimal or figure out the statements of the lemmas that you need to show for the correctness.

   (d) Analyze the time complexity of your implementation.

5. Consider the problem of making change for n cents using the minimum number of coins. Assume all coin denominations are positive integers.

   (a) Suppose the available coins are quarters (25), dimes (10), nickels (5), and pennies (1).
   - Describe a greedy algorithm to make change.
   - Prove that your algorithm always produces an optimal solution.

   (b) Suppose the available coin denominations are powers of c: $c^0, c^1, c^2, \ldots, c^k$, for integers $c > 1$ and $k \geq 1$. Prove that the greedy algorithm always yields an optimal solution.

   (c) Give an example of a set of coin denominations (including a penny) for which the greedy algorithm does not produce an optimal solution. Justify your answer with a counterexample.

(d) Design a dynamic programming algorithm that computes the minimum number of coins needed to make change for n cents using any set of k denominations (assuming one denomination is 1). Analyze the time complexity of your algorithm.

6. You are given a set of n tasks $S = \{a_1, a_2, \ldots, a_n\}$, where each task a_i requires p_i units of processing time. Let C_i denote the completion time of task a_i. The objective is to minimize the average completion time: $\frac{1}{n}\sum_{i=1}^{n} C_i$.

   (a) Assume all tasks are available at time 0, and once a task starts, it must run nonpreemptively until completion.
   average completion time. Also prove that your algorithm is optimal and analyze its running time.

   (b) Now suppose each task a_i has a release time r_i, and tasks may be preempted and resumed later.

   For the above settings, design a greedy algorithm that schedules the tasks to minimize the average completion time. Also prove that your algorithm is optimal and analyze its running time.

7. Given n items where item i has weight w_i and value v_i, and a knapsack with capacity W: answer the following.
   - Design a DP algorithm to maximize total value without exceeding capacity.
   - Define the DP state and recurrence.
   - Analyze time and space complexity.

8. Given two strings X and Y, compute the minimum number of insertions, deletions, and substitutions needed to transform X into Y. Write the DP recurrence and analyze time complexity.

9. Given a string of length n, design a DP algorithm to find the length of the longest palindromic subsequence. Define subproblems clearly and analyze time complexity.

10. Given an $m \times n$ grid of non-negative integers: find a path from top-left to bottom-right minimizing the sum. Write the DP recurrence and analyze time complexity.

11. Given an integer x and a positive integer n, design a divide and conquer algorithm to compute x^n efficiently. Write the recurrence for the running time and analyze the time complexity.

12. Given an array of n numbers, design a divide and conquer algorithm to count the number of inversions (pairs (i, j) such that $i < j$ and $A[i] > A[j]$). Analyze the time complexity of your algorithm.

13. Given two sorted arrays of size n, design a divide and conquer algorithm to find the median of the combined $2n$ elements in $O(\log n)$ time.

14. Describe the divide and conquer strategy for solving the Towers of Hanoi puzzle with n disks. Write the recurrence relation for the number of moves and solve it.

15. Given a partially filled 9×9 Sudoku grid: design a backtracking algorithm to complete the grid. Explain how constraints (row, column, 3x3 box) are used for pruning.

16. Given an $m \times n$ maze with blocked and open cells: design a backtracking algorithm to find all paths from the start to the destination.

17. Given n jobs with profits and deadlines: use branch and bound to schedule jobs to maximize total profit. Define the state space tree and the bounding function.

18. Given a cost matrix for assigning n tasks to n agents: design a branch and bound method to find the assignment with minimum cost. Discuss how to compute bounds for partial assignments.

19. Given an $m \times n$ grid of letters and a dictionary of words: design a backtracking algorithm to find all dictionary words present in the grid by moving horizontally, vertically, or diagonally. Discuss pruning strategies to reduce unnecessary searches.

20. Given a graph $G = (V, E)$: design a backtracking algorithm to find a Hamiltonian path (visits every vertex exactly once) or cycle. Discuss pruning strategies to improve efficiency.